
Temporal Mixture-of-Experts: Serving Memory that Scales with Active Parameters

Noah Cylich
noahcylich@gmail.com

Abstract

Mixture-of-Experts (MoE) models improve language model quality by activating only a small fraction of parameters per token, but serving them still requires the full expert pool in memory. We study *temporal MoE*, which enforces expert reuse across neighboring tokens so serving memory can scale with active rather than total parameters. Unlike post-hoc caching or offloading, we impose locality as a training-time constraint. A rolling-residency router keeps only the k active experts of each layer resident and swaps at most one expert per token. Across FLAME-MoE isoFLOP sweeps spanning 10^{16} to 10^{19} FLOPs, temporal MoE recovers 72–82% of the MoE-over-dense quality gain at compute-optimal sizes, and zero-shot downstream accuracy at 10^{19} tracks the recovery. The fine-grained configuration holds 18 of 192 experts resident, a $5.7\times$ cut in whole-model weight memory. A llama.cpp implementation serves an $\sim 11\text{B}$ -scale model at $0.83\times$ the all-resident decode speed in $5.1\times$ less VRAM. The trained router internalizes the constraint. Expert specialization shifts from token identity to surrounding context, and temporal self-consistency nearly doubles over vanilla routing. Trained-in locality makes MoE-quality models practical to serve in the memory of their active parameters.

1 Introduction

Sparse MoE layers buy quality with total parameters while paying compute only for the k experts active per token [23, 3, 9]. Serving inverts the bargain. In FLAME-MoE-1.7B [11], routed experts hold 91% of all weights, yet every expert must sit in fast memory even though most do nothing for a given token. Offloading systems stream experts from slower storage on demand [4, 25], but per-token offload is bandwidth-infeasible. Hiding one expert miss costs roughly the fast-to-slow bandwidth ratio, 20–110 resident-expert computations on modern hardware, while only $k \approx 6\text{--}8$ experts of work exist per layer to hide it behind. Caching lowers the miss rate, not the miss cost, and vanilla routers give it little to work with, as consecutive tokens share only about 20% of their expert choices [9, 19, 15]. The locality that serving needs must be trained in, not recovered after the fact.

We contribute (i) a rolling-residency MoE trained from scratch under a hard residency constraint with an unmodified loss stack, (ii) isoFLOP evidence at 10^{16} , 10^{17} , 10^{18} , and 10^{19} FLOPs that it recovers most of the MoE quality gain at a fraction of the resident memory, and that zero-shot task accuracy at 10^{19} tracks the pretraining recovery, (iii) a mechanistic account of how the router internalizes the constraint, a shift of expert selection off token identity and onto context (Section 4), together with measured negative results that redirect future work from cache policy toward anticipation, and (iv) a real-engine llama.cpp serving measurement that validates the memory-latency claim.

2 Method

Each MoE layer with E routed experts keeps only its k active experts resident, the set S_t . Given router logits $R_t \in \mathbb{R}^E$ for token t , the layer nominates the best non-resident expert, $n_t = \arg \max_{e \notin S_{t-1}} R_t[e]$, and swaps it in only if it beats the worst resident, which is evicted,

$$S_t = \begin{cases} (S_{t-1} \setminus \{\arg \min_{e \in S_{t-1}} R_t[e]\}) \cup \{n_t\}, & R_t[n_t] > \min_{e \in S_{t-1}} R_t[e], \\ S_{t-1}, & \text{otherwise.} \end{cases} \quad (1)$$

The token is served by S_t with gates from the softmax of resident logits, and S_0 cold-fills from the first token’s top- k . Training uses the standard language modeling, load-balancing, and z-losses unchanged. The swap is non-differentiable, but the router receives ordinary gate gradients on every resident it scores, so the function the swap rule reads is trained normally. The selection scan runs as a single-launch Triton kernel and costs about 1 ms per layer call, under 1% of step time at our scales.

The single-swap budget is what lets the load hide behind compute. At small batch the expert FFN is memory-bandwidth-bound, so serving the $k - 1$ already-resident experts reads $(k - 1)b$ bytes from fast memory while the incoming expert streams its b bytes from slower storage, and the load is fully masked when $b/\text{BW}_{\text{slow}} \leq (k - 1)b/\text{BW}_{\text{fast}}$, that is, once $k - 1$ approaches the fast-to-slow bandwidth ratio (roughly 32 for RAM over NVMe). The expert size b cancels, so the condition depends only on the routing width, which DeepSeek-style fine-graining toward $k \approx 32$ reaches without changing active parameters or FLOPs, the knob granularity scaling laws quantify [14, 2, 17]. With 18 of 192 experts resident (the same fraction as 6 of 64 in the coarse model) the routed-expert footprint shrinks $10.7\times$ and whole-model resident memory $5.7\times$. Section 3 tests these calculations in a real engine, including a variant that computes the router earlier in the network than its experts to lengthen the masking window.

We build on FLAME-MoE [11] (64 experts, top-6, one shared expert) in two versions, a coarse-expert model (6 active of 64 experts) and a fine-grained model (18 of 192, DeepSeek-style splitting [3]). At 10^{16} and 10^{17} FLOPs we sweep isoFLOP curves over 0.77M–15.1M active non-embedding parameters with a 16k vocabulary. At 10^{18} we sweep three sizes around an exact FLAME-MoE-38M-100M replication (4.45B tokens, 50k pythia vocabulary). We gave the compute-optimal 38M midpoint three seeds per arm (Table 6), the flanks one. At 10^{19} we train the budget’s compute-optimal shape and evaluate it zero-shot on six standard tasks with lm-evaluation-harness. For serving, we implement rolling residency in a llama.cpp CUDA fork and benchmark batch-1 prefill and decode on Qwen3-MoE-shaped random-weight models ($\sim 11.3\text{B}$ total, Q4_K_M, both granularities scaled $\sim 10\times$), with $R = k$ expert slots held in VRAM against a full CPU pool and at most one swap per layer per token. Prefill runs expert-major, and each needed expert streams once through a fixed ring of slots and serves all of its tokens in one batched pass. Weights are irrelevant to a latency and memory measurement. All runs use locked hyperparameters and report test-set bits-per-byte (BPB, tokenizer-invariant, seed noise ≈ 0.003). Each shape gets a dense floor matched to the MoE’s active parameters and FLOPs, and we define *recovery* as $(\text{dense} - \text{temporal})/(\text{dense} - \text{MoE})$.

3 Results

IsoFLOP bands. At every model size in the three budgets with full size sweeps (Figure 1), temporal MoE sits well inside the dense-to-MoE band and recovers 72–82% of the MoE-over-dense gain at compute-optimal sizes (72% and 77% for the fine-grained model, 82% coarse) with only k experts ever resident. The isoFLOP parabola and its minimum match the dense and MoE baselines at every budget, so the constraint does not distort scaling geometry.

Larger budgets. At 10^{18} FLOPs (Figure 1, bottom left) recovery is 78% against the strongest full MoE, consistent with the smaller budgets. Sometimes the mechanism appears nearly lossless, here outperforming its equally fine-grained MoE counterpart (seed means 1.334 vs. 1.348 BPB). At the coarse granularity temporal holds parity with the strongest full MoE (1.312 vs. 1.317). Seed spread within every arm stays under 0.004 BPB (Table 6), in general our BPB results have low variance. Fine-graining itself hurts the full MoE at this scale (at 48.5M active it lands within 0.008 BPB of the dense floor) while temporal routing is robust to it, so the only fine-grained model we take to 10^{19} is the temporal one. At 10^{19} (Figure 1, bottom right), our largest budget, the temporal models keep 78–81% of the MoE-over-dense gain and the granularity preference flips: the fine-grained temporal model (1.065 BPB) now edges out the coarse one (1.068).

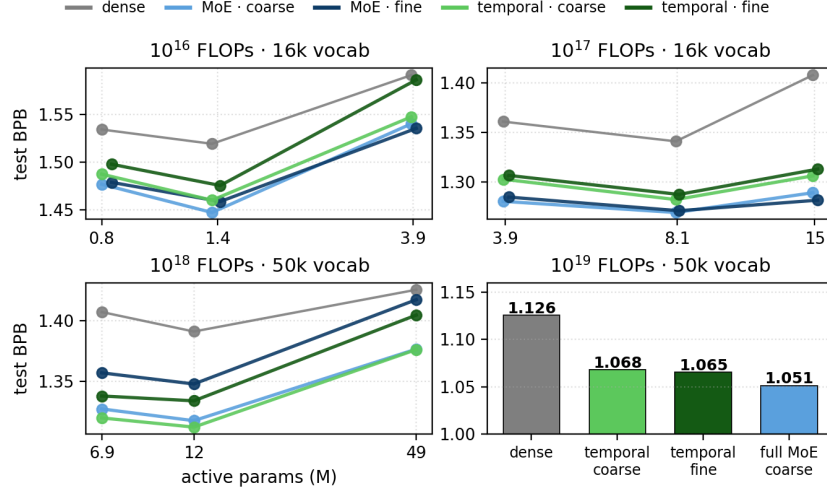


Figure 1: Quality at fixed compute, one panel per budget (test-set BPB, lower is better). Shade encodes granularity, coarse 6-of-64 normal and fine 18-of-192 dark. Different vocabularies mean BPB is comparable within a panel, not across. Temporal MoE lands inside the dense-to-MoE band at every size and budget and keeps 72–82% of the MoE-over-dense gain. At 10^{18} the coarse temporal model reaches parity with the coarse full MoE. Its midpoint plots three-seed means (Table 6).

Table 1: Zero-shot downstream accuracy at 10^{19} FLOPs (lm-evaluation-harness, acc_norm except WinoGrande which reports acc). Avg-chance subtracts each task’s chance floor (0.5 for WinoGrande and PIQA, 0.25 elsewhere) before averaging. Bold = best per column. Per-task standard errors are 0.005–0.020 (largest on OpenBookQA), so single-task bold margins of a point or two (WinoGrande, ARC-c) are within noise and the aggregate columns carry the comparison.

model	OBQA	WinoG	PIQA	ARC-e	ARC-c	HSwag	Avg	Avg-chance
dense	.276	.501	.621	.437	.222	.296	.392	.059
full MoE, 6 of 64	.296	.507	.659	.495	.247	.341	.424	.091
temporal, 6 of 64	.292	.512	.659	.473	.246	.336	.420	.086
temporal, 18 of 192	.288	.487	.652	.481	.254	.336	.416	.083

Downstream tasks. Zero-shot accuracy at 10^{19} (Table 1) tracks the pretraining picture. The full MoE lifts the dense average from 0.392 to 0.424 (0.059 to 0.091 above chance), and the temporal models keep 86% (coarse) and 76% (fine) of that above-chance gain. The dense-to-MoE average gap spans several standard errors while the MoE-to-temporal gaps sit within one, so the lift over dense is statistically clean and the residual temporal deficit is not resolved at this scale. On WinoGrande and PIQA the coarse temporal model matches the full MoE. Recovery transfers from pretraining loss to task accuracy.

Serving. Table 2 closes the loop in a real engine. The deploy row serves the fine-grained model at $0.83\times$ the all-resident decode speed in $5.1\times$ less VRAM. The measured $5.1\times$ is the end-to-end counterpart of the $5.7\times$ weight arithmetic in Section 2, and the gap is the KV cache and workspace buffers, which sit in VRAM regardless of residency. The other rows isolate two costs. Kernel overhead is granularity-independent (0.86 – $0.88\times$). The streaming cost is set by granularity, roughly free for fine experts (840 KiB per swap, hidden behind compute) but dominant for coarse ones (2.5 MiB, $0.51\times$). The vanilla-offload row shows the same budget without the constraint. Free routing misses most of its experts every token and runs 3 – $4\times$ slower than deploy at the same VRAM. The asymmetry is the measured converse of the 10^{19} crossover. Scale pushes temporal quality toward fine granularity, and fine granularity is what streams cheaply, so quality and serving select the same design point. The router-early variant lengthens the masking window and recovers most of the fine model’s remaining gap, as the bandwidth analysis anticipates. Figure 2 extends the measurement to prefill and long context. Prefilling 16k tokens takes $1.6\times$ the all-resident time and decode holds near $0.80\times$ throughout. Peak VRAM stays under 3.6 GiB against 8.0–9.3 GiB all-resident, a gap the

Table 2: Real-engine decode throughput (tok/s at 1024-token context, llama.cpp CUDA, RTX A6000) for the $\sim 11.3\text{B}$ -total models of Section 2. Deploy holds only the $R = k$ active experts in VRAM (1502 vs 7672 MiB, a $5.1\times$ total and $10.6\times$ expert-tier reduction) and streams the rest from a CPU pool. The vanilla-offload row is the same memory budget without the trained constraint, free top- k routing with every miss fetched, measured at the $\sim 20\%$ consecutive overlap of released MoEs (Appendix B).

setup	fine 18-of-192	vs ceiling	coarse 6-of-64	vs ceiling
all experts in VRAM (ceiling)	200	$1.00\times$	252	$1.00\times$
our kernel, no swaps ($p = 0$)	176	$0.88\times$	217	$0.86\times$
deploy: active-only in VRAM	165	$0.83\times$	128	$0.51\times$
router-early variant	173	$0.87\times$	143	$0.57\times$
vanilla offload: active-only, free routing	43	$0.22\times$	42	$0.17\times$

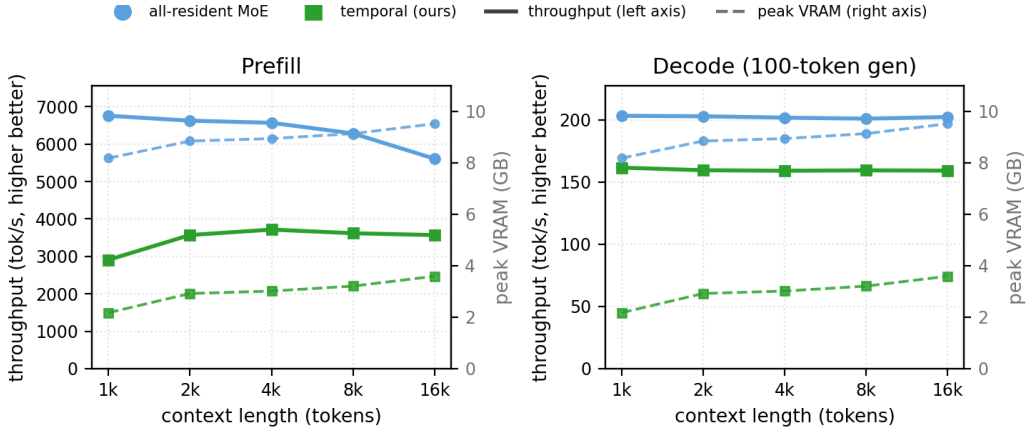


Figure 2: Serving vs. context length (fine-grained model). Solid = throughput (left axis), dashed = peak VRAM (right axis). Decode runs after the full prefill, slightly below Table 2’s decode-only setting.

growing KV cache compresses from $3.7\times$ at 1k to $2.6\times$ at 16k. At larger batch each sequence carries its own resident set, so the advantage narrows there too (protocol and kernel details in Appendix B).

The constraint is internalized. Figure 3 shows why this works. A router trained under residency shows temporally banded *unconstrained* preferences, and its next-token self-consistency nearly doubles over vanilla routing at every scale probed. The effect is streaming, not collapse. Over a 2048-token sequence a temporal model still touches 83–98% of its expert pool (96–100% at 10^{19}), and no expert is resident more than 40% of the time (25% at 10^{19}), so quality comes from time-multiplexed access to the full pool rather than a static subset.

4 De-lexicalization of routing

Section 3 showed that the router internalizes the constraint. This section asks what it learns instead of the free routing it can no longer express. We analyze trained temporal and full-MoE pairs fine-grained (18 of 192) at 10^{16} and coarse (6 of 64) at 10^{17} FLOPs, and replicate every analysis at 10^{19} , where the flip is cleanest, with no full-MoE expert context-dominated against 88–96% of temporal experts. The full MoE holds a small quality edge in every analyzed pair, so what follows characterizes what the constraint changes, not whether it helps. Protocols and full tables are in Appendix A.

Usage concentration. We measure each expert’s usage breadth as the effective fraction of evaluation tokens it serves, the normalized inverse Simpson concentration of its routing mass, which is 1 under uniform usage and near 0 when usage concentrates on a few recurring tokens. In the fine-grained pair, the median full-MoE expert serves an effective 30% of the token stream and almost no expert serves a majority of it. The median temporal expert serves 66%, two thirds of temporal experts serve a majority, and the coarse pair shifts the same way. Expert weights, in contrast, are nearly unaffected, equally distinct and near orthogonal in both regimes. Independent seeds and a sigmoid-router control reproduce every gap (Appendix A). The constraint therefore changes how experts are used rather than what they compute, and the rest of this section identifies what replaces token ownership.

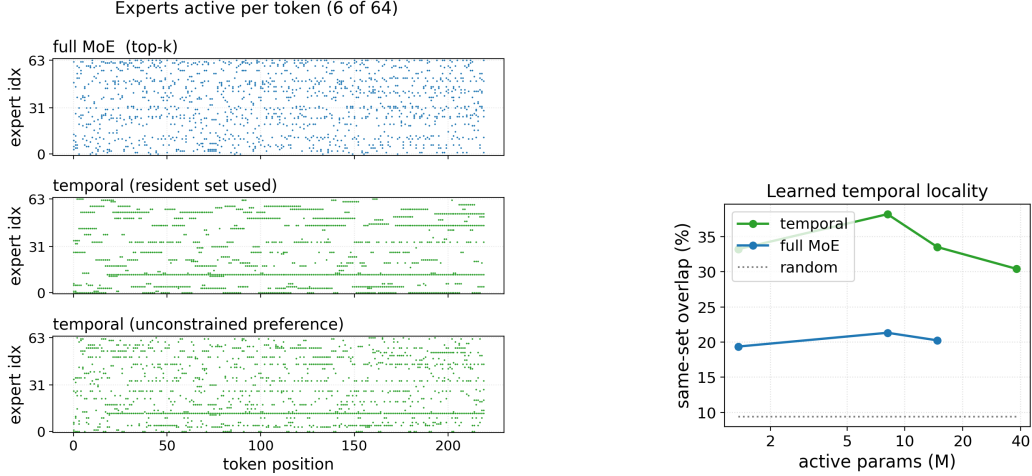


Figure 3: **The router learns the locality it is constrained to.** Left, experts active per token (8.1M model, deepest MoE layer). The vanilla MoE (top) scatters, the temporal resident set (middle) forms long streaks, and the temporal router’s *unconstrained* preference (bottom, mask removed) is also streaked, so the model wants stable experts rather than merely tolerating them. Right, overlap between a token’s free top- k and the previous active set, 30–38% for temporal vs. 19–21% vanilla and 9.4% random, flat from 1.4M to 38M active parameters.

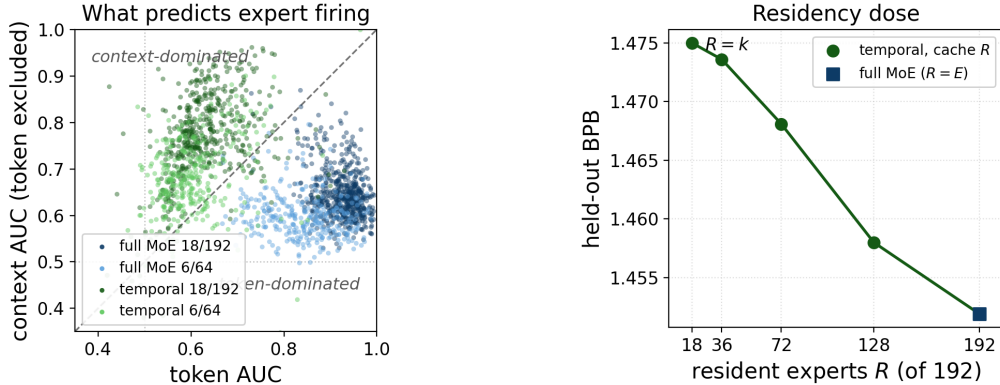


Figure 4: **Left**, one point per expert, by how well the current token (x) versus the surrounding context excluding it (y) predicts its firing (held-out AUC, chance 0.500, window = residency lifetime, colors as in Figure 1). Full-MoE experts are token lookups, temporal experts context-dominated. **Right**, held-out BPB vs. cache size R at identical FLOPs (192 experts, $k = 18, 10^{16}$). The constraint is a tunable quality cost, and because R is the resident-expert memory, the curve is the serving memory-quality frontier.

Token to context. For each expert, two logistic probes predict its firing, one from the current token’s embedding alone and one from the surrounding window with the token excluded, sized to one residency lifetime, the number of active experts. Figure 4 (left) plots every expert. Full-MoE routing is close to a token lookup. The token alone predicts firing at median AUC 0.93, and context dominates for under 1% of experts. Temporal routing inverts the picture. Token AUC falls to 0.62, and context dominates for 85–91% of experts. We call this *de-lexicalization*. The shift is visible forward in time: a probe predicting the router’s next-token expert selection from the preceding tokens’ routing record alone reaches AUC 0.85 on temporal models against 0.64 for the full MoE at matched granularity (0.93–0.98 against 0.60 at 10^{19}), because context persists across neighboring tokens and token identity does not. A data-weighted logit lens [20] agrees on the output side, where full-MoE experts promote coherent word clusters and temporal experts show no detectable vocabulary preference.

Constraint strength. Although residency restricts the router relative to its unconstrained preferences, each model performs best under its training regime: unmasking a temporal model and imposing

residency on a full MoE both degrade quality (Table 4), so the routing organization is trained into the weights rather than removable at inference. The unmask penalty grows monotonically with budget, +0.10 BPB at 10^{16} to +0.21 at 10^{19} , so the organization deepens with scale. The constraint’s strength is also a continuous dial. Decoupling the cache size R from the active k interpolates between our deployed setting ($R = k$) and the full-MoE recipe ($R = E$) at identical FLOPs, and held-out BPB falls monotonically as the cache grows (Figure 4, right), so at this budget the constraint is a pure, tunable quality cost. R is the number of experts held in fast memory, so the curve is the serving memory-quality frontier. The maximal constraint costs +0.023 BPB at roughly a tenth of the routed-expert memory.

Expert conditioning. Usage uniformity carries into the statistics of what the experts learn. At 10^{19} the temporal model’s routed experts are less heavy-tailed than the full MoE’s in both weights (max excess kurtosis 0.57 vs 0.97) and FFN activations (median 0.31 vs 0.46). The payoff is quantization. At 3-bit precision the temporal model loses 0.038 CE where the full MoE loses 0.045, and the ordering holds at every bit width, granularity, and scale we measured. Per-expert weight norms keep the same spread in both regimes, while the largest weight entries shrink at 10^{18} and the typical expert’s peak activation drops by a third to a half at both budgets (Appendix D).

A free router binds experts to token identities. Rolling residency makes that binding unrepresentable while leaving weights untouched, so the router reorganizes around context, which persists across a residency lifetime. That reorganization explains how quality survives the constraint, the near-doubled self-consistency of Figure 3, and the forecastable demand that the anticipation direction of Section 6 targets.

5 Related work

Serving systems keep a pool of $R > k$ experts in fast memory and manage the misses. Eliseev and Mazur cache experts with LRU and speculative prefetch [4], Pre-gated MoE computes each layer’s gate one layer early so the fetch overlaps compute [8], MoE-Infinity prefetches from activation traces [29], ProMoE trains a predictor to fetch ahead of the router [27], and a survey covers the rest of a crowded field [16]. Skliar et al. instead bias a frozen router toward whatever is cached [25]. A newer line shapes the routing itself. Oracle-MoE routes in a compact semantic space derived from attention so that consecutive tokens pick similar experts [31], Sticky Routing and BlockFFN add losses that penalize switches or shrink the union of experts per chunk [12, 26], CoSMoEs’ block-wise selection loss buys fewer swaps at a measured quality cost [7], router-only finetuning raises reuse in released checkpoints [32, 21], and a learned controller can decide when a checkpoint’s expert set switches [24]. Across both lines locality is a tendency. The pool still provisions for misses, so the footprint remains a knob priced in latency or quality at serving time. Rolling residency makes the footprint and the swap budget constraints the model trains under, and that is what lets the bandwidth condition of Section 2 hold by construction rather than in expectation.

A complementary axis serves capacity whose lookup key is known before any compute runs. Mixture of Lookup Experts trains experts that read the token embedding, then re-parameterizes them into offloaded tables fetched by token id [10]. Per-layer embeddings ship the same mechanism in Gemma’s edge models [5], and Engram scales it to N-gram-keyed conditional memory [1]. Lookup capacity is lexical by construction, and Section 4 shows token-identity binding is exactly what a free router spends experts on and what residency removes. The two axes therefore split the problem instead of competing. Tables can hold the token-keyed capacity, residency streams the context-dependent experts no table can express, and the combination is natural and untested.

Constraints on routing structure exist along other axes and for other goals. Path-constrained MoE ties expert choices across neighboring layers [6], and Mixture-of-Depths spends its sparsity on skipping compute across the sequence [22]. Residency is the time-axis member of this family, aimed at serving memory because that is the cost that binds at deployment.

6 Conclusion and next steps

Across four compute decades, two tokenizer families, and two expert granularities, training under rolling residency keeps 72–82% of the MoE-over-dense quality gain with only the active experts resident. It preserves the baselines’ scaling geometry, and the recovery carries to downstream tasks at

10^{19} . A real-engine implementation serves the fine-grained model at $0.83\times$ the all-resident decode speed in $5.1\times$ less VRAM. Quality survives because the router reorganizes. Routing de-lexicalizes, and experts bind to context that persists across a residency lifetime (Section 4).

Our negative results narrow down where the remaining deficit can be closed (Appendix E). A coherence loss that imitates past routing hurts at every weight. Better eviction is worth little, while a future-demand oracle gains 18–30 points of hit rate. Two objectives that reward the router’s own predictability both Goodhart (one pins an expert, the other collapses diversity). The headroom is anticipation, and it must be decoupled from the router’s objective.

Two directions follow. The first is to adapt a released MoE checkpoint to residency with router-only training (zero-shot imposition is the costliest direction in Table 4), which would make temporal routing a serving recipe for existing MoE-class models in the memory of their active parameters. The second is behavior under reinforcement learning, where routing flips between the rollout engine and the trainer are the identified collapse driver [18, 30]. Residency cuts both ways there. The resident set is extra state the trainer must replay, but the single-swap budget bounds routing drift to one expert per layer per token, a bound vanilla routing does not have. The rest of the stability profile is measured. Appendix D finds smoother weights and activations than the full MoE’s, no pinned experts, and better low-bit quantization.

The end goal is local inference. If temporal routing holds at scale, a Kimi-K2-scale sparse model [13] could run in the memory of its active parameters on quantized consumer hardware. That would put near-state-of-the-art models within private, local reach of the average person.

References

- [1] X. Cheng et al. Conditional memory via scalable lookup: a new axis of sparsity for large language models. *arXiv:2601.07372*, 2026.
- [2] A. Clark et al. Unified scaling laws for routed language models. *arXiv:2202.01169*, 2022.
- [3] D. Dai et al. DeepSeekMoE: towards ultimate expert specialization in mixture-of-experts language models. *arXiv:2401.06066*, 2024.
- [4] A. Eliseev and D. Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv:2312.17238*, 2023.
- [5] Gemma Team. Gemma 4 technical report. *arXiv:2607.02770*, 2026.
- [6] Z. Gu et al. Path-constrained mixture-of-experts. *arXiv:2603.18297*, 2026.
- [7] P. Huber et al. CoSMoEs: compact sparse mixture of experts. *arXiv:2503.00245*, 2025.
- [8] R. Hwang et al. Pre-gated MoE: an algorithm-system co-design for fast and scalable mixture-of-expert inference. *arXiv:2308.12066*, 2023.
- [9] A. Q. Jiang et al. Mixtral of experts. *arXiv:2401.04088*, 2024.
- [10] S. Jie et al. Mixture of lookup experts. *arXiv:2503.15798*, 2025.
- [11] H. Kang, Z. Yu, and C. Xiong. FLAME-MoE: a transparent end-to-end research platform for mixture-of-experts language models. *arXiv:2505.20225*, 2025.
- [12] A. Kayyam. Sticky routing: training MoE models for memory-efficient inference. *arXiv:2607.08780*, 2026.
- [13] Kimi Team. Kimi K2: open agentic intelligence. *arXiv:2507.20534*, 2025.
- [14] J. Krajewski et al. Scaling laws for fine-grained mixture of experts. *arXiv:2402.07871*, 2024.
- [15] J. Liang et al. Not all models suit expert offloading: on local routing consistency of mixture-of-expert models. *arXiv:2505.16056*, 2025.
- [16] J. Liu et al. A survey on inference optimization techniques for mixture of experts models. *arXiv:2412.14219*, 2024.

- [17] J. Ludziejewski et al. Joint MoE scaling laws: mixture of experts can be memory efficient. *arXiv:2502.05172*, 2025.
- [18] W. Ma et al. Stabilizing MoE reinforcement learning by aligning training and inference routers. *arXiv:2510.11370*, 2025.
- [19] N. Muennighoff et al. OLMoE: open mixture-of-experts language models. *arXiv:2409.02060*, 2024.
- [20] nostalgebraist. Interpreting GPT: the logit lens. LessWrong, 2020.
- [21] A. Raje, A. Nayak, and G. Joshi. MELINOE: fine-tuning enables memory-efficient inference for mixture-of-experts models. *arXiv:2602.11192*, 2026.
- [22] D. Raposo et al. Mixture-of-depths: dynamically allocating compute in transformer-based language models. *arXiv:2404.02258*, 2024.
- [23] N. Shazeer et al. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. *arXiv:1701.06538*, 2017.
- [24] Z. Shen and P. Henderson. Temporally extended mixture-of-experts models. *arXiv:2604.20156*, 2026.
- [25] A. Skliar et al. Mixture of cache-conditional experts for efficient mobile device inference. *arXiv:2412.00099*, 2024.
- [26] C. Song et al. BlockFFN: towards end-side acceleration-friendly mixture-of-experts with chunk-level activation sparsity. *arXiv:2507.08771*, 2025.
- [27] X. Song et al. ProMoE: fast MoE-based LLM serving using proactive caching. *arXiv:2410.22134*, 2024.
- [28] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv:2408.15664*, 2024.
- [29] L. Xue et al. MoE-Infinity: efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv:2401.14361*, 2024.
- [30] C. Zheng et al. Group sequence policy optimization. *arXiv:2507.18071*, 2025.
- [31] J. Zhou et al. Oracle-MoE: locality-preserving routing in the oracle space for memory-constrained large language model inference. In *ICML*, 2025.
- [32] X. Zhu et al. ReMoE: boosting expert reuse through router fine-tuning in memory-constrained MoE LLM inference. *arXiv:2605.27081*, 2026.

A De-lexicalization details

All statistics in Section 4 derive from router probabilities, expert weight matrices, and expert output vectors on a fixed evaluation batch of 64 sequences of 2048 tokens per model.

Selectivity and weight identity. Renormalizing expert e ’s gate mass over token positions gives $q_e(t)$, and selectivity is the normalized inverse Simpson concentration $1/(N \sum_t q_e(t)^2)$, the effective fraction of the stream feeding the expert. Generalists are experts above 0.5, and router flatness is per-token entropy normalized by $\ln E$. Weight identity flattens each expert’s FFN weights and reports cosine distance to the expert-weight centroid and median pairwise cosine. Table 3 gives all cells. Independent seed replicates agree to about 0.01 in selectivity and two points in generalist fraction, and the regime gaps are ten to forty times larger.

Firing probes. For each expert, the label marks whether it is in the token’s top- k . Two logistic classifiers predict the label from the model’s own input embeddings, the current token alone versus the mean embedding of tokens within window w with the current token excluded. The exclusion prevents context features from encoding the token itself. Classifiers fit on the first 70% of tokens

Table 3: Routing structure and weight identity across all cells: median selectivity, generalist fraction, normalized router entropy, mean cosine distance to the expert-weight centroid, and median pairwise cosine.

model	selectivity	generalist %	entropy	centroid dist	pairwise cos
full MoE, 18 of 192	0.30	1.7	0.88	0.90	0.005
sigmoid control (2 seeds)	0.27–0.28	5	0.87	0.89–0.90	0.007
temporal, 18 of 192 (2 seeds)	0.65–0.66	65–67	0.95	0.87–0.88	0.010–0.012
full MoE, 6 of 64	0.34	13	0.86	0.84	0.009
temporal, 6 of 64	0.52	54	0.93	0.84	0.012
full MoE, 6 of 64, 10^{19}	0.27	24	0.85	0.87	0.002
temporal, 6 of 64, 10^{19}	0.60	68	0.95	0.86	0.004
temporal, 18 of 192, 10^{19}	0.51	53	0.95	0.91	0.003

and report AUC on the last 30%. Refitting on null labels, both iid permutations and circular shifts of at least 1000 tokens (preserving residency-induced autocorrelation), gives 0.500 ± 0.002 at the small budgets. At 10^{19} the iid null stays at 0.500–0.502 while the shift null reads 0.502–0.505 and rises with temporality, the preserved residency autocorrelation itself, a floor common to both probes. At $w = k$, median token AUC / context AUC / fraction context-dominated are 0.93 / 0.64 / 1% (full MoE, 192 experts), 0.62 / 0.77 / 91% (temporal, 192), 0.84 / 0.59 / 1% (full MoE, 64), and 0.60 / 0.68 / 85% (temporal, 64), consistent at every MoE layer (89–92% context-dominated by depth for the fine temporal model). At 10^{19} on the same protocol the medians are 0.845 / 0.607 / 0% (full MoE, 64), 0.611 / 0.672 / 88% (temporal, 64), and 0.559 / 0.633 / 96% (temporal, 192). Across $w \in \{k/2, k, 32\}$ the fine pair stays context-dominated (88–94%), and the coarse pair is context-dominated at both residency-scale windows, washing out only at $w = 32$, five times its cache lifetime. The demand-prediction probe is causal and history-only: features at token t are the current gate values and a short demand history (lags t to $t-3$ and fast and slow exponential averages), with no hidden states, and the label is whether an expert is in the router’s selection at the next token. AUC is 0.85–0.87 across temporal models and 0.64 for the full MoE at matched granularity. At 10^{19} it is 0.926 (coarse) and 0.979 (fine) against 0.604.

Output lens. Averaging each expert’s output vectors over the tokens actually routed to it and projecting through the final norm and unembedding gives the vocabulary distribution the expert promotes. Its exponentiated entropy is an effective vocabulary between 1 and the 16k vocabulary size. Projecting without data conditioning reads about 15,990 for every expert, the method’s no-signal reference. Full-MoE experts reach median 14,612 with a sharpest decile of 9,683, whose top promoted tokens read as coherent lexical clusters. Temporal experts sit at median 15,932, with no word-list experts even in the tail. At 10^{19} (50k vocabulary) the gap widens, weighted median effective vocabulary 4,421 for the full MoE against 7,438 and 9,168 for temporal, with the static no-signal reference reading $\sim 30k$ at this scale.

Table 4: Constraint swap. Unmasking makes every expert resident for a trained temporal model. Imposing puts a trained full MoE under rolling residency. Held-out BPB except the 10^{18} row, which is validation CE in nats on the analysis batch (its native value accordingly differs from that cell’s seed-1 test CE, 3.909 nats or 1.313 BPB in Figure 1, by 0.006). Imposing costs two to five times more than unmasking, the asymmetry expected if full-MoE experts are token-bound: a token denied its bespoke expert has nowhere to go, while a contextual expert serves its neighborhood regardless. The fine-grained full-MoE row is the sigmoid-router control, whose artifacts were preserved. For the gradient control, top- k routing fixes each expert’s expected share of the batch in any regime, and measured tokens per expert per update are identical across regimes (12,288 fine, 3,072 coarse).

trained model	swap	native	swapped	Δ
temporal, 18 of 192, 10^{16}	unmask	1.4750	1.5744	+0.10
temporal, 6 of 64, 10^{17}	unmask	1.2821	1.4063	+0.12
temporal, 6 of 64, 10^{18}	unmask	3.9037	4.3890	+0.49
temporal, 6 of 64, 10^{19}	unmask	1.0675	1.2739	+0.21
temporal, 18 of 192, 10^{19}	unmask	1.0649	1.2655	+0.20
full MoE, 18 of 192, 10^{16}	impose	1.4499	1.6902	+0.24
full MoE, 6 of 64, 10^{17}	impose	1.2690	1.8789	+0.61
full MoE, 6 of 64, 10^{19}	impose	1.0510	1.4824	+0.43

Dose sweep. Every dose is trained from scratch at 10^{16} FLOPs on the same softmax-aux recipe. Held-out BPB at $R = 18/36/72/128/192$ is 1.4750 / 1.4736 / 1.4681 / 1.4580 / 1.4519 (endpoint = the retrained full-MoE baseline). Most of the gap closes by $R = 128$, and expert diversity is preserved at every dose (182–186 effective experts).

B Serving details

All measurements use the llama.cpp CUDA fork of Section 2 on an RTX A6000 (Q4_K_M, fine-grained model), four trials with the first discarded, decode over a 100-token generation after the prefill. Full rows live in `results/ablations/serving_benchmarks.csv`.

Micro-batch choice. llama.cpp processes the prompt in micro-batches (its ubatch). Table 5 sweeps the size at 16k context. Figure 2 runs both tiers at 2048, within 1% of the all-resident time optimum and near the deploy VRAM floor, so the setting flatters neither side. Larger micro-batches cost the deploy tier much of its memory advantage, since activation buffers grow with tokens in flight, and smaller ones stall it.

Table 5: Prefill micro-batch sweep at 16k context, fine-grained model. Time is prefill in ms (lower is better), memory is peak VRAM in MiB. Bold = the operating point of Figure 2.

micro-batch	all-resident		deploy	
	ms	MiB	ms	MiB
1024	2975	8894	5527	3159
2048	2942	9512	4499	3590
4096	2922	10746	4132	4986
16384	4723	18162	4247	13358

Where prefill time goes. Three configurations at 16k in one micro-batch decompose the cost. The stock all-resident kernel takes 4715 ms, the expert-major kernel 4219 ms with all experts resident and 4297 ms streaming from the CPU pool. Streaming is nearly free at every context ($1.00\text{--}1.05\times$ the resident control), so the deploy cost is the kernel shape, $2.4\times$ the stock kernel at 1k and faster than it at 16k in one micro-batch. A wave-batched `mul_mat_id` prototype that avoids the expert-major reorder was a measured no-go.

The vanilla-offload floor. Figure 5 sweeps decode speed against pinned cold expert fetches per layer per token, measured with real PCIe transfers on a reconstruction of the serving models (the recipe and scripts are committed, and the rebuilt fine model reproduces the ceiling at 200.8 tok/s and the recorded KV growth). Free top- k routing at active-only memory misses $k(1 - k/E) \approx 16$ experts per layer at chance overlap and about 14 at the $\sim 20\%$ consecutive overlap of released MoEs, which lands at 39–43 tok/s. Eviction picks the lowest-index non-selected slot, a choice that is byte-neutral for a timing measurement. Points at two or fewer fetches are launch-bound and carry boost variance. The bandwidth-bound floor is stable.

C Seed replication at 10^{18} FLOPs

Table 6 lists the runs behind the 10^{18} midpoint of Figure 1.

Table 6: Per-seed test BPB at the 10^{18} 38M midpoint, all on the same data split. The dense floor is a single run.

model	seed			mean
	1234	2	3	
temporal, 6 of 64	1.3128	1.3111	1.3128	1.3122
full MoE, 6 of 64	1.3158	1.3197	1.3169	1.3175
temporal, 18 of 192	1.3354	1.3339	1.3323	1.3339
full MoE, 18 of 192	1.3461	1.3489	1.3483	1.3478
dense	1.3911			1.3911

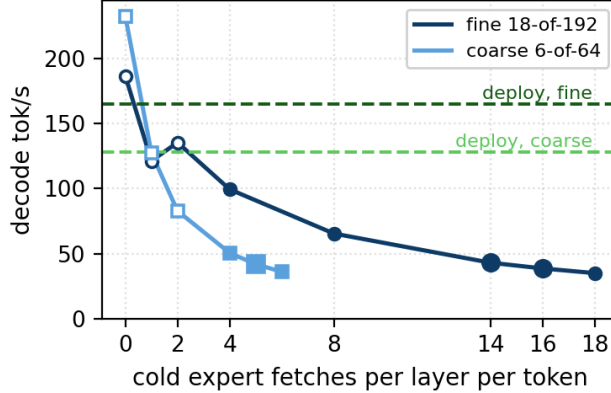


Figure 5: Decode speed vs pinned cold expert fetches per layer per token. Hollow markers are launch-bound points with boost variance, enlarged markers are the rates behind Table 2’s vanilla-offload row. Dashed lines are the temporal deploy rows at the same VRAM.

D Weight and activation stability

All statistics derive from the trained checkpoints and one forward pass per model over the fixed evaluation batch of Appendix A, with the 10^{19} cells primary and the 10^{18} 38M cells as replication. Excess kurtosis is 0 for a Gaussian and grows with tail weight.

Weights and activations. Table 7 collects the per-expert statics. The kurtosis gaps of Section 4 replicate and widen at 10^{18} , the spread of per-expert weight norms is unchanged (coefficient of variation 0.39–0.49 in every cell and both regimes), the largest weight entries shrink at 10^{18} and match at 10^{19} , and the typical expert’s peak intermediate activation drops by a third to a half. One tail event runs the other way, the single largest activation at 10^{19} coarse is higher under temporal (41 vs 19) against a lower typical level. Peak attention logits stay at 32–39 against the dense model’s 38. Router row norms are uncorrelated with each expert’s residency frequency (correlations -0.27 to $+0.03$ across cells), and the served set tracks the router’s unconstrained top- k within a mean gap of 0.011–0.021, so the resident-only gate gradient leaves no trace in the router. Figure 6 shows the 10^{19} training curves. The dense baseline has one mid-run loss spike (to 2.61 BPB at iteration 2300, recovered within about 200 iterations) and coarse temporal a smaller, briefer one (to 1.51 at iteration 830, recovered within tens). The full-MoE and fine temporal runs are smooth, no run diverged, and gradient norms are spike-free at 10^{18} .

Table 7: Per-expert statics. Weight kurtosis is the max over experts and matrices, $\max |w|$ the largest entry, activation kurtosis and peak activation the medians over experts (FFN intermediates). Dense FFN kurtosis is ≈ 0 . Lower is smoother throughout.

budget	model	weight kurt	$\max w $	act kurt	act peak
10^{19}	temporal, 6 of 64	0.57	0.21	0.31	5.6
10^{19}	full MoE, 6 of 64	0.97	0.21	0.46	8.3
10^{19}	temporal, 18 of 192	1.50	0.24	0.29	6.5
10^{18}	temporal, 6 of 64	0.66	0.19	0.21	3.0
10^{18}	full MoE, 6 of 64	2.63	0.24	1.02	5.1
10^{18}	temporal, 18 of 192	2.00	0.22	0.38	3.8
10^{18}	full MoE, 18 of 192	6.01	0.28	1.04	8.0

Quantization. Figure 7 applies per-group round-to-nearest (group 128) to routed-expert weights only and re-evaluates test CE. 8-bit is lossless everywhere. At 4 and 3 bits the temporal model degrades less than its full-MoE pair at both granularities and both scales, and the 10^{19} models are two to three times more robust than the 38M ones. Smoother weights are cheaper to quantize, and the deploy stack of Section 3 already runs at Q4_K_M.

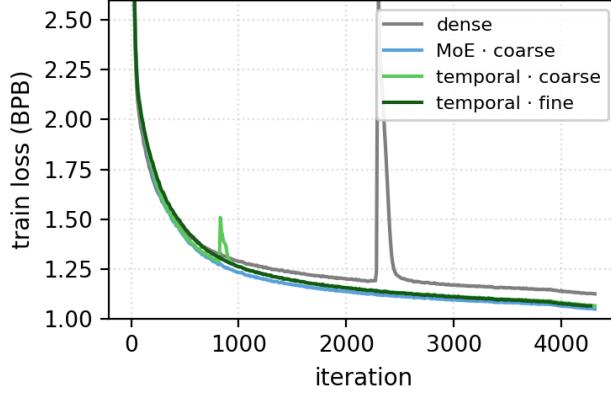


Figure 6: Training loss at 10^{19} (BPB, every 10 iterations). The dense spike at iteration 2300 peaks at 2.61 (clipped) and recovers. Coarse temporal’s smaller excursion at 830 recovers within tens of iterations. No run diverged.

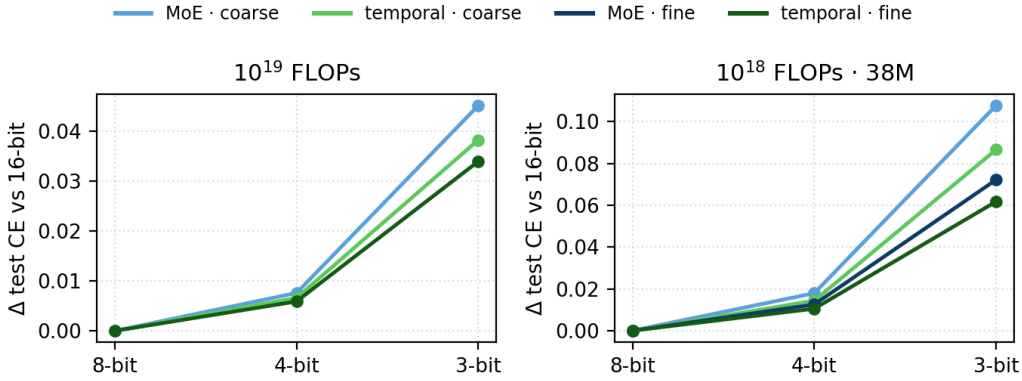


Figure 7: Test-CE cost of quantizing routed-expert weights (per-group RTN, group 128), lower is better. Panels have independent scales. Temporal sits below its matched full MoE at every point.

E Negative results

Coherence loss. An explicit loss that pulls router logits toward the current resident set hurts monotonically at every weight scanned at 10^{16} , across both granularities. The constraint is already internalized, and a backward-looking gradient adds no new information.

Eviction replay. Replaying logged routing through alternative eviction shows the offline-optimal (Belady) policy is worth only 6–10 points of set hit rate over our least-logit rule, and LRU is 4–8 points worse, while an oracle that selects swaps by short-horizon *future* demand gains 18–30 points. Cache policy is nearly saturated. Anticipation is not.

Predictability objectives. Two objectives that reward the router’s own predictability both Goodhart at sweep budgets. Auxiliary-loss-free, DeepSeek-inspired selection biases [28] with demand momentum raise residency alignment (+7 points of next-token overlap at the best swept setting) but induce a de-facto pinned expert, resident 85% of the time, with no quality gain. An anticipatory loss on discounted future demand raises alignment sharply (+26 points) by teaching the router to concentrate demand so its own future is easy to predict. Streamed expert diversity collapses threefold (expert union 158 to 50 of 192) and BPB worsens. A published loss of this family, CoSMoEs’ block-wise expert selection [7], reports the same trade, fewer expert replacements for a consistent quality regression.